

CSC 466: Knowledge Discovery in Data

Cal Poly, San Luis Obispo

Spring 2026

Lab 0

Due Tuesday, April 14, 3:00pm

Before You Do Anything Else!!!

This is the Instructor's version of this notebook. **You need to save a copy of this notebook** to your personal Google Drive. To do this, go to File > Save a copy in Drive.

Name: Nathan Madlansacay

Cal Poly Email: nmadlans@calpoly.edu

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import math
import time
### import specific sklearn classes below

from sklearn import datasets
from sklearn.metrics.pairwise import euclidean_distances
```

Instructions

Develop and test all functionality needed to complete **Lab 0** in the code and Markdown cells below as specified in the lab assignment. Use as many code/Markdown cells as you need.

```
In [2]: ## Loading the iris dataset
iris = datasets.load_iris()
```

```
In [3]: iris_array = iris["data"]
df_iris = pd.DataFrame(iris_array)
df_iris
```

```
Out[3]:
```

	0	1	2	3
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

```
In [4]: ### Loading the California housing dataset
housing = datasets.fetch_california_housing()
```

```
In [11]: housing['data']
df_housing = pd.DataFrame(housing['data'])
df_housing = df_housing.head(5000)
```

Version 1:

Distance between points (Manhattan)

```
In [28]: def manhattan_distance(row_x, row_b):
          return sum(abs(a - b) for a, b in zip(row_x, row_b))
```

PairWise Function

```
In [29]: def pairwise_distance(df):
          records = df.values.tolist()
          ind = list(df.index)
          n = len(records)
          distance = {}
          most_similar = (None, None, float('inf'))
          least_similar = (None, None, float('-inf'))

          for i in range(n):
              for j in range(i + 1, n):
                  dist = manhattan_distance(records[i], records[j])
                  distance[(ind[i], ind[j])] = dist
```

```

        if dist < most_similar[2]:
            most_similar = (ind[i], ind[j], dist)

        if dist > least_similar[2]:
            least_similar = (ind[i], ind[j], dist)

    return most_similar, least_similar

```

```

In [30]: def report_results(df, most_similar, most_dissimilar, elapsed):
        i_sim, j_sim, d_sim = most_similar
        i_dis, j_dis, d_dis = most_dissimilar

        print(f"\nMost SIMILAR -> rows {i_sim} & {j_sim}")
        print(f"Distance: {d_sim:.6f}")
        print(f"Row{i_sim:>3}: {df.loc[i_sim].tolist()}")
        print(f"Row{j_sim:>3}:{df.loc[j_sim].tolist()}")

        print(f"\nMost DISSIMILAR -> rows {i_dis} & {j_dis}")
        print(f"Distance{d_dis:.6f}")
        print(f"Row{i_dis:>3}:{df.loc[i_dis].tolist()}")
        print(f"Row{j_dis:>3}:{df.loc[j_dis].tolist()}")

        print(f"Time elapsed:{elapsed:.6f} seconds")

```

RESULTS

```

In [31]: most_similar_iris, least_similar_iris = pairwise_distance(df_iris)
        start = time.perf_counter()
        most_similar_iris, least_similar_iris = pairwise_distance(df_iris)
        end = time.perf_counter()
        elapsed = end - start

        report_results(df_iris, most_similar_iris, least_similar_iris, elapsed)

```

Most SIMILAR -> rows 101 & 142

Distance: 0.000000

Row101: [5.8, 2.7, 5.1, 1.9]

Row142:[5.8, 2.7, 5.1, 1.9]

Most DISSIMILAR -> rows 22 & 118

Distance12.100000

Row 22:[4.6, 3.6, 1.0, 0.2]

Row118:[7.7, 2.6, 6.9, 2.3]

Time elapsed:0.075960 seconds

```

In [32]: start = time.perf_counter()
        most_similar, least_similar = pairwise_distance(df_housing)
        end = time.perf_counter()
        elapsed = end - start

        report_results(df_housing, most_similar, least_similar, elapsed)

```

Most SIMILAR -> rows 904 & 931

Distance: 0.862480

Row904: [5.388, 26.0, 5.772277227722772, 0.9183168316831684, 1285.0, 3.1806930693069306, 37.54, -121.99]

Row931:[4.9464, 26.0, 5.689655172413793, 0.9787798408488063, 1285.0, 3.4084880636604775, 37.54, -122.04]

Most DISSIMILAR -> rows 922 & 1914

Distance12368.165623

Row922:[8.4045, 7.0, 7.63523372061605, 1.0440421507700621, 12203.0, 3.2972169683869224, 37.53, -121.92]

Row1914:[1.875, 33.0, 141.9090909090909, 25.636363636363637, 30.0, 2.727272727272727, 38.91, -120.1]

Time elapsed:84.387749 seconds

RESULTS: It ran the code but it was taking forever to calculate the distances for each variable within each 20,000 different rows. Using at least 5000 rows instead of 20000

Version 2:

In [33]: **def** cos_similarity_V2(df):

 X = df.values

 dot_product = X @ X.T

 norms = np.sqrt((X ** 2).sum(axis=1))

 norm_matrix = norms[:, np.newaxis] * norms[np.newaxis, :]

 similarity_matrix = dot_product / norm_matrix

return similarity_matrix

In [34]: **def** pairwise_distance_V2(df):

 n = df.shape[0]

 dist_masked = euc_distance_V2(df)

 np.fill_diagonal(dist_masked, np.inf) *#used to ignore rows that are comparing t*

 min_idx = np.unravel_index(np.argmin(dist_masked), dist_masked.shape) *#reshapes*

 np.fill_diagonal(dist_masked, 0)

 max_idx = np.unravel_index(np.argmax(dist_masked), dist_masked.shape)

return min_idx, max_idx

In [35]: **def** report_results_V2(df, dist_matrix, min_idx, max_idx, elapsed):

 i_sim, j_sim = min_idx

 i_dis, j_dis = max_idx

 print(f"\n Most SIMILAR -> rows {i_sim} & {j_sim}")

 print(f"Distance: {dist_matrix[i_sim, j_sim]:.6f}")

 print(f"Row{i_sim:<3}:{df.iloc[i_sim].tolist()}")

 print(f"Row{j_sim:<3}:{df.iloc[j_sim].tolist()}")

```

print(f"\n Most DISSIMILAR -> rows {i_dis} & {j_dis}")
print(f"Distance:{dist_matrix[i_dis, j_dis]:.6f}")
print(f"Row {i_dis:<3}:{df.iloc[i_dis].tolist()}")
print(f"Row {j_dis:<3}:{df.iloc[j_dis].tolist()}")

print(f"Time elapsed : {elapsed:.6f} seconds")

```

Results

```

In [36]: start = time.perf_counter()

dist_matrix = cos_similarity_V2(df_iris)
min_idx, max_idx = pairwise_distance_V2(df_iris)

end = time.perf_counter()
elapsed = end - start

report_results_V2(df_iris, dist_matrix, min_idx, max_idx, elapsed)

```

Most SIMILAR -> rows 101 & 142
 Distance: 1.000000
 Row101:[5.8, 2.7, 5.1, 1.9]
 Row142:[5.8, 2.7, 5.1, 1.9]

Most DISSIMILAR -> rows 13 & 118
 Distance:0.833869
 Row 13 :[4.3, 3.0, 1.1, 0.1]
 Row 118:[7.7, 2.6, 6.9, 2.3]
 Time elapsed : 0.102145 seconds

```

In [37]: start = time.perf_counter()

dist_matrix = cos_similarity_V2(df_housing)
min_idx, max_idx = pairwise_distance_V2(df_housing)

end = time.perf_counter()
elapsed = end - start

report_results_V2(df_housing, dist_matrix, min_idx, max_idx, elapsed)

```

Most SIMILAR -> rows 16 & 544
 Distance: 1.000000
 Row16 :[2.775, 52.0, 5.9395770392749245, 1.0483383685800605, 793.0, 2.395770392749245, 37.85, -122.27]
 Row544:[2.9732, 52.0, 5.585284280936455, 1.1705685618729098, 793.0, 2.652173913043478, 37.77, -122.26]

Most DISSIMILAR -> rows 922 & 3126
 Distance:0.059164
 Row 922:[8.4045, 7.0, 7.63523372061605, 1.0440421507700621, 12203.0, 3.2972169683869224, 37.53, -121.92]
 Row 3126:[2.375, 4.0, 1.0, 1.0, 6.0, 3.0, 35.21, -117.79]
 Time elapsed : 6.130411 seconds

The code also runs but there isn't enough ram to run the whole numpy array onto the data set. (Tried both on Jupyter *my ram* and Collab Notebook which both ended in a crash as well.

```
In [20]: def pairwise_distance_V3(df):

    dist_matrix = euclidean_distances(df)
    dist_df = pd.DataFrame(dist_matrix, index=df.index, columns=df.index)

    np.fill_diagonal(dist_df.values, np.inf)
    min_val = dist_df.min().min()
    min_idx = dist_df.stack().idxmin()

    np.fill_diagonal(dist_df.values, 0)
    max_val = dist_df.max().max()
    max_idx = dist_df.stack().idxmax()

    return min_val, min_idx, max_val, max_idx
```

```
In [21]: def report_results_V3(df, min_val, min_idx, max_val, max_idx, elapsed):
    i_min, j_min = min_idx
    i_max, j_max = max_idx

    print(f"\nMost SIMILAR -> rows {i_min} & {j_min}")
    print(f"Distance: {min_val:.6f}")
    print(f"Row {i_min:<3}: {df.loc[i_min].tolist()}")
    print(f"Row {j_min:<3}: {df.loc[j_min].tolist()}")

    print(f"\nMost DISSIMILAR -> rows {i_max} & {j_max}")
    print(f"Distance: {max_val:.6f}")
    print(f"Row {i_max:<3}: {df.loc[i_max].tolist()}")
    print(f"Row {j_max:<3}: {df.loc[j_max].tolist()}")

    print(f"Time elapsed : {elapsed:.6f} seconds")
```

```
In [22]: start = time.perf_counter()
min_val, min_idx, max_val, max_idx = pairwise_distance_V3(df_iris)
end = time.perf_counter()
elapsed = end - start
report_results_V3(df_iris, min_val, min_idx, max_val, max_idx, elapsed)
```

Most SIMILAR -> rows 101 & 142

Distance: 0.000000

Row 101: [5.8, 2.7, 5.1, 1.9]

Row 142: [5.8, 2.7, 5.1, 1.9]

Most DISSIMILAR -> rows 13 & 118

Distance: 7.085196

Row 13 : [4.3, 3.0, 1.1, 0.1]

Row 118: [7.7, 2.6, 6.9, 2.3]

Time elapsed : 0.057550 seconds

```
In [23]: start = time.perf_counter()
min_val, min_idx, max_val, max_idx = pairwise_distance_V3(df_housing)
```

```
end = time.perf_counter()
elapsed = end - start
report_results_V3(df_housing, min_val, min_idx, max_val, max_idx, elapsed)
```

Most SIMILAR -> rows 16 & 544

Distance: 0.501986

Row 16 : [2.775, 52.0, 5.9395770392749245, 1.0483383685800605, 793.0, 2.395770392749245, 37.85, -122.27]

Row 544: [2.9732, 52.0, 5.585284280936455, 1.1705685618729098, 793.0, 2.652173913043478, 37.77, -122.26]

Most DISSIMILAR -> rows 922 & 3126

Distance: 12197.004588

Row 922: [8.4045, 7.0, 7.63523372061605, 1.0440421507700621, 12203.0, 3.2972169683869224, 37.53, -121.92]

Row 3126: [2.375, 4.0, 1.0, 1.0, 6.0, 3.0, 35.21, -117.79]

Time elapsed : 1.697844 seconds

Congratulations! You are done with Lab 0

```
In [ ]: # @markdown Run this cell to download this notebook as a webpage, `_NOTEBOOK.html`.

import google, json, nbformat

# Get the current notebook and write it to _NOTEBOOK.ipynb
raw_notebook = google.colab._message.blocking_request("get_ipynb",
                                                    timeout_sec=30)["ipynb"]
with open("_NOTEBOOK.ipynb", "w", encoding="utf-8") as ipynb_file:
    ipynb_file.write(json.dumps(raw_notebook))

# Use nbconvert to convert .ipynb to .html.
!jupyter nbconvert --to html --log-level WARN _NOTEBOOK.ipynb

# Download the .html file.
google.colab.files.download("_NOTEBOOK.html")
```

- Open `_NOTEBOOK.html` in your browser, and save it as a PDF.
 - Go to File > Print > Save as PDF.
- Double check that all of your code and output is visible in the saved PDF.
- Upload the PDF to Gradescope.
 - Please be sure to select the correct pages corresponding to each question.